

HTTP 请求与参数位置

适用：Python 3.10+、Pydantic 2，以及支持 Annotated 的 FastAPI。示例采用现代写法；核对日期：2026-09-14。

请求行说明操作和目标，请求头提供附加信息，请求体承载提交的数据。Path、Query 负责声明参数来源与校验；Field 负责描述模型字段。

```
PUT /items/123?notify=true HTTP/1.1
Host: localhost:8001
Authorization: Bearer example-token
Content-Type: application/json

{"name": "Keyboard", "price": 99.9}
```

内容	HTTP/1.1 中的位置	FastAPI 对应
路径参数 123	请求行的 /items/123	Path(); 路由 /items/{item_id}
查询参数 notify=true	请求行中 ? 后面的部分	Query()
Authorization	请求头	Header()
Cookie	请求头的 Cookie 字段	Cookie()
name、price	空行之后的 JSON 请求体	BaseModel + Field()
表单 / 上传文件	请求体	Form() / File() / UploadFile

先记住这四个问题

Path：操作哪个对象？例如商品 123。Query：怎样筛选或处理？例如分页、分类、排序。Header：携带哪些附加信息？例如身份凭证。Body：实际提交什么数据？例如商品名称和价格。

一个请求可以同时使用这些来源。查询参数不限于 GET；POST、PUT 也可以带查询参数。HTTP/2、HTTP/3 不使用同样的文本请求行，但这些概念仍然存在。

FastAPI 常见推断规则

名称出现在路由占位符中：路径参数。普通标量参数且不在路径中：通常是查询参数。BaseModel 类型参数：通常是请求体。需要明确来源时使用 Path、Query、Body 等；BaseModel 也可以通过 Query 显式绑定为查询参数模型。

官方：请求体与参数识别

Path()：路径参数

从 fastapi 导入 Path。它为路由里的占位符增加校验与文档说明。路径参数始终必填，不能给它设置可省略的默认值。

```
from typing import Annotated
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/items/{item_id}")
def read_item(
    item_id: Annotated[int, Path(ge=1, le=100000,
                                description="Item ID")],
):
    return {"item_id": item_id}
```

常用参数	作用	示例
gt / ge	大于 / 大于等于	gt=0; ge=1
lt / le	小于 / 小于等于	lt=100; le=100
multiple_of	必须是指定数值的倍数	multiple_of=5
min_length / max_length	字符串长度下限 / 上限	min_length=2, max_length=30
pattern	正则约束字符串；完整匹配需加锚点	pattern=r"^[a-z0-9-]+\$"
title / description	文档标题 / 说明，不负责验证值	description="商品编号"
examples	文档中的示例值列表	examples=[123]
deprecated	将参数标为已弃用，不会禁用接口	deprecated=True

数字约束如何选择

gt=0 表示不能等于 0；ge=0 表示可以等于 0。lt 和 le 同理。数值约束用于数字字段，长度和 pattern 约束用于字符串等适用类型，不要混用。

访问结果

/items/123: 通过。/items/0: 小于下限，默认返回 422。/items/abc: 不能解析成整数，默认返回 422。
/items/: 不匹配这个路由，通常是 404，而不是“缺少参数”的 422。

旧式等价写法: item_id: int = Path(..., ge=1)。其中 ... 表示必填，并不是默认值。Annotated 写法中无需写这个省略号。

官方: [Path 与数值校验](#)

Query()：查询参数

从 fastapi 导入 Query。查询参数来自 URL 的 ? 后面，多个键值对用 & 连接。默认值决定它能否省略。

```
from typing import Annotated
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items")
def list_items(
    q: Annotated[str | None, Query(max_length=50)] = None,
    page: Annotated[int, Query(ge=1)] = 1,
    size: Annotated[int, Query(ge=1, le=100)] = 20,
    tags: Annotated[list[str] | None, Query()] = None,
):
    return {"q": q, "page": page, "size": size, "tags": tags}
```

常用参数 / 写法	作用	示例
默认值	Annotated 写法中放在 = 后面	page: Annotated[int, Query(ge=1)] = 1
gt / ge / lt / le	数值范围	Query(ge=1, le=100)
min_length / max_length	字符串长度；列表类型可限制元素数量	Query(min_length=2, max_length=50)
pattern	字符串正则约束	Query(pattern=r"^(asc desc)\$")
alias	外部参数名与 Python 名称不同	Query(alias="item-query")
title / description	接口文档说明	Query(description="搜索关键词")
examples	文档示例列表，不提供默认值	Query(examples=["phone"])
deprecated	文档中标记弃用	Query(deprecated=True)
include_in_schema	是否出现在 OpenAPI 文档中	Query(include_in_schema=False)

访问 /items?q=phone&page=2&tags=a&tags=b，会将 tags 解析为 ["a", "b"]。?tags=a,b 不会自动拆成两个标签。

必填关键词: q: Annotated[str, Query(min_length=1)]，不写默认值。可省略关键词: q: Annotated[str | None, Query()] = None。

官方: [Query](#)、[字符串约束](#)、[别名和列表](#)

BaseModel 与 Field()

BaseModel 描述一组数据；Field() 为其中一个字段设置默认值、限制或文档信息。两者都从 pydantic 导入。Field 本身不指定 HTTP 来源，也不是数据库字段声明。

常用参数	作用	示例
default	字段默认值；没有默认值通常必填	Field(default=0)
default_factory	每次创建实例时生成默认值；与 default 二选一	Field(default_factory=list)
gt / ge / lt / le	数值范围	Field(gt=0, le=9999)
multiple_of	数值必须为指定数的倍数	Field(multiple_of=5)
min_length / max_length	字符串长度或集合元素数量	Field(min_length=1, max_length=50)
pattern	字符串正则约束	Field(pattern=r"^[A-Z] {3}\$")
max_digits / decimal_places	Decimal 的总位数 / 小数位数限制	Field(max_digits=8, decimal_places=2)
title / description / examples	字段的文档说明与示例	Field(examples=["Keyboard"])
alias	输入别名；输出别名需启用对应序列化设置	Field(alias="itemName")
validation_alias / serialization_alias	分别设置输入名与输出名	Field(serialization_alias="itemName")
strict	禁用该字段的部分自动类型转换	Field(strict=True)
validate_default	校验字段默认值，默认不校验	Field(default=1, ge=1, validate_default=True)
exclude / repr	序列化时排除 / 对象文字表示时隐藏	Field(exclude=True, repr=False)
frozen	限制实例创建后对该字段重新赋值	Field(frozen=True)

alias 的输出行为要区分：普通 Pydantic model_dump() 使用 model_dump(by_alias=True) 才明确按别名输出；FastAPI 响应模型另有自己的配置。

exclude=True、repr=False 不是加密；frozen=True 也不保证列表等内部对象完全不可变。查询参数来自 URL 文本，初学时不要随意给 int 查询参数加 strict=True。

官方：Pydantic 字段、默认值、别名和约束

官方：在请求体模型中使用 Field

完整示例：三种参数一起使用

将下面代码保存到练习项目的主文件 `main.py`。它同时演示 `Path`、`Query`、`Header` 和请求体中的 `Field`。

```
from typing import Annotated
from fastapi import FastAPI, Header, Path, Query
from pydantic import BaseModel, Field

app = FastAPI()

class ItemUpdate(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    price: float = Field(gt=0, description="Unit price")
    tags: list[str] = Field(default_factory=list, max_length=5)
    note: str | None = Field(default=None, max_length=200)

@app.put("/items/{item_id}")
def update_item(
    item_id: Annotated[int, Path(ge=1)],
    item: ItemUpdate,
    notify: Annotated[bool, Query()] = False,
    authorization: Annotated[str | None, Header()] = None,
):
    return {
        "item_id": item_id,
        "notify": notify,
        "item": item.model_dump(),
    }
```

注意：这里只读取 `Authorization` 请求头，并没有实现身份认证；读取一个头字段不等于验证登录。

启动与测试

```
python -m uvicorn main:app --reload --port 8001
```

打开 `http://127.0.0.1:8001/docs`，展开 `PUT` 接口，点击 `Try it out`。填写 `item_id=123`、`notify=true`，请求体如下：

```
{
  "name": "Keyboard",
  "price": 99.9,
  "tags": ["office"],
  "note": null
}
```

输入会被解析为：`item_id` 是 `int`，`notify` 是 `bool`，`item` 是 `ItemUpdate` 实例。通过 `item.name` 读取字段，通过 `item.model_dump()` 转成字典。

把 `price` 改为 0、`name` 改为空字符串，或者删除必填 `price`，默认会得到 422 校验错误。自定义异常处理可以改变返回格式。

必填、可空与常见误区

Pydantic 2 字段写法	可以省略?	可以传 null?
name: str	否	否
name: str = "guest"	是	否
name: str None	否	是
name: str None = None	是	是
name: str = Field(min_length=1)	否	否

“允许 None”和“可以不传”不是同一个概念。对于 JSON 请求体，null 对应 Python 的 None；URL 中 ?q=null 通常只是字符串 "null"。

常见错误速查

容易写错的地方	正确理解
PATH() / QUERY() / field()	这里的名字是 Path()、Query()、Field()，区分大小写。
在 Annotated 内写 Query(default=1)	默认值写在函数参数 = 右边，不要两处同时设置。
Field 放在路由函数里代替 Query	Field 用于模型字段；函数参数来源用 Path、Query、Body 等声明。
设置 examples 就有默认值	examples 是文档展示；默认值需要 default 或 =。
字符串用 ge，数字用 min_length	数字使用范围限制；字符串使用长度或 pattern。
Pydantic 2 仍使用 regex	新代码使用 pattern；regex 是旧教程常见写法。
改默认值或说明没更新 /docs	确认浏览器端口、服务重载，并刷新页面；只改返回内容不一定改变文档。
Field 能检查所有业务条件	跨字段关系用 model_validator；单字段复杂逻辑用 field_validator。

一个与 RAG 项目有关的例子

chunk_size 和 chunk_overlap 都是整数，可以分别用 Field 限制范围；但“重叠必须小于分块大小”需要跨字段校验。只给它们分别设置 ge、le，不能保证两者的关系正确。

继续查阅：官方参考

- FastAPI - 请求参数完整参考 (Path / Query / Header / Body 等)
 - FastAPI - 请求体与字段
 - Pydantic - 字段校验与模型校验器
- 本笔记整理常用参数，并非全部 API。优先掌握：参数来源、默认值、ge/gt/le/lt、长度、pattern、alias 和 Field(default_factory=...)。